

期末專題：Small-C 互動式解譯器

作業說明文件

項目	說明
課程名稱	系統軟體 (System Software)
專題性質	期末實作專題 (個人或至多三人一組)
公布週次	第十週
繳交期限	第十六週上課日
實作語言	Python 3 (3.10 以上版本)

一、作業目標

本專題要求設計並實作一個 Small-C 語言的互動式解譯器 (interactive interpreter)。此解譯器應提供類似早期 BASIC 解譯器的操作環境：使用者可在命令提示符下逐行輸入 Small-C 程式碼，由解譯器即時解析並執行；亦可透過內建指令載入完整的原始碼檔案後一次執行。

本專題的設計意圖在於延伸並整合課堂所學的系統軟體核心技術——包括符號表管理、語彙分析 (lexical analysis)、語法導向處理 (syntax-directed processing)、以及執行期環境的模擬——同時引導學生跨越從組合語言處理器到高階語言處理器的認知門檻。學生在 SIC/XE 組譯器中已經練習過的 token 辨識、符號表建構與查詢、錯誤偵測與回報等技術，均可直接遷移到本專題的開發過程中。

選用 Python 3 作為實作語言的考量在於：Python 豐富的字串處理能力與動態資料結構可以讓學生將精力集中於解譯器的核心邏輯設計，而非記憶體管理等底層細節。此外，Python 本身即為直譯式語言，以直譯式語言來建構另一個直譯式語言的過程，有助於學生深刻理解解譯器的運作原理。

二、Small-C 語言規範

本專題所定義的 Small-C 為標準 C 語言 (ISO C) 的嚴格子集，其設計參考 Ron Cain (1980) 與 James Hendrix (1984) 所定義的 Small-C 語言，但為配合教學目的進行了若干調整與明確化。以下規範為本專題解譯器所必須支援的完整語言定義，任何未在此規範中列出的 C 語言特性，學生均不需實作。

2.1 資料型別

Small-C 僅支援以下基本資料型別：

`int`：有號整數，寬度為 4 個位元組（32 位元），數值範圍為 -2,147,483,648 至 2,147,483,647。

`char`：有號字元，寬度為 1 個位元組（8 位元），數值範圍為 -128 至 127，同時可用於儲存 ASCII 字元。

`int*` 與 `char*`：分別為指向 `int` 與 `char` 的指標型別。指標值在解譯器內部以模擬位址（整數偏移量）表示。

Small-C 不支援 `float`、`double`、`long`、`short`、`unsigned`、`void`（但 `void` 可出現於函式回傳型別，表示無回傳值）、`struct`、`union`、`enum`、`typedef`，亦不支援多層指標（如 `int**`）。

2.2 常數與字面值

整數常數支援十進位表示法（如 `42`、`-7`）與十六進位表示法（以 `0x` 或 `0X` 為前綴，如 `0xFF`）。

字元常數以單引號包圍（如 `'A'`、`'0'`），並支援以下跳脫序列：`\n`（換行）、`\t`（水平定位）、`\0`（空字元）、`\\`（反斜線）、`\'`（單引號）、`\"`（雙引號）。

字串常數以雙引號包圍（如 `"hello\n"`），在記憶體中以 `char` 陣列方式儲存，並自動附加結尾的空字元 `\0`。字串常數僅可出現於函式呼叫的引數位置（如 `printf("value = %d\n", x);`）以及 `char` 陣列的初始化中。

2.3 變數宣告

全域變數（global variables）與區域變數（local variables）的宣告語法相同：

```
int x;
int y = 10;
char ch = 'A';
int arr[20];
char str[80];
int *ptr;
char *cp;
```

全域變數宣告於所有函式之外，其生命週期為整個程式執行期間。區域變數宣告於函式本體的起始處（所有可執行語句之前），其生命週期為該函式的單次呼叫期間。Small-C 不支援區塊內部的變數宣告（即不允許在 `if`、`while` 等區塊內部宣告新變數）。

一維陣列的大小必須以整數常數指定，不支援可變長度陣列。陣列的索引以 0 為起始。

2.4 運算子

依優先順序由高至低排列如下（同一層級內由左至右結合，除非特別標註為右結合）：

第 1 級（最高）：後綴運算子 `()` 函式呼叫、`[]` 陣列索引。

第 2 級：前綴一元運算子 `-`（負號）、`!`（邏輯否定）、`~`（位元補數）、`*`（指標取值）、`&`（取址）、`++`（前綴遞增）、`--`（前綴遞減）。此級為右結合。

第 3 級：乘法類 `*`（乘）、`/`（除）、`%`（餘數）。

第 4 級：加法類 `+`（加）、`-`（減）。

第 5 級：位移運算子 `<<`、`>>`。

第 6 級：關係運算子 `<`、`<=`、`>`、`>=`。

第 7 級：相等運算子 `==`、`!=`。

第 8 級：位元 AND `&`。

第 9 級：位元 XOR `^`。

第 10 級：位元 OR `|`。

第 11 級：邏輯 AND `&&`（支援短路求值）。

第 12 級：邏輯 OR `||`（支援短路求值）。

第 13 級（最低）：指定運算子 `=`、`+=`、`-=`、`*=`、`/=`、`%=`。此級為右結合。

後綴 `++` 與 `--` 不列入必要支援範圍，學生可自行選擇是否實作。

2.5 控制結構

Small-C 支援以下控制結構，其語意與標準 C 相同：

`if` 語句與選擇性的 `else` 子句。支援 `else if` 的鏈式寫法。

`while` 迴圈。

`for` 迴圈，包含初始化表達式、條件表達式與迭代表達式三個部分。

`do / while` 迴圈。

`break` 語句，用於跳出最內層的 `while`、`for` 或 `do / while` 迴圈。

`continue` 語句，用於跳至最內層迴圈的下一次迭代。

`return` 語句，用於從函式返回，可附帶回傳值（`return expr;`）或不帶回傳值（`return;`）。

`switch / case` 為選擇性實作的加分項目，不列為必要要求。

2.6 函式

函式定義的語法如下：

```
int add(int a, int b) {  
    return a + b;  
}  
  
void greet() {  
    printf("Hello!\n");  
}
```

函式的回傳型別可為 `int`、`char` 或 `void`。參數傳遞方式為傳值呼叫 (call by value)，指標參數可用於模擬傳址效果。函式支援遞迴呼叫。所有函式必須在呼叫之前完成定義 (不支援前向宣告)。

每個可執行程式必須包含一個名為 `main` 的函式作為程式進入點，其宣告形式為 `int main()` 或 `void main()`。在互動模式下，若使用者輸入的語句不在任何函式定義之內，解譯器應在全域環境中直接執行該語句。

2.7 內建函式

解譯器必須提供以下內建函式，供 `Small-C` 程式直接呼叫。所有內建函式無需事先宣告即可使用。

輸入與輸出函式

`int putchar(int ch)`：輸出一個字元至標準輸出，回傳該字元的整數值。

`int getchar()`：從標準輸入讀取一個字元，回傳其整數值；若遇到輸入結束，回傳 `-1`。

`void printf(char *fmt, ...)`：格式化輸出函式，必須支援的格式碼包括 `%d` (十進位整數)、`%c` (字元)、`%s` (字串)、`%x` (十六進位整數)、`%%` (輸出百分號字元本身)。不需支援欄位寬度、精確度或對齊旗標等進階格式化功能。

`void puts(char *s)`：輸出一個字串並自動換行。

`int scanf(char *fmt, ...)`：格式化輸入函式，必須支援的格式碼包括 `%d` (讀取整數) 與 `%c` (讀取字元)。引數必須為指標 (使用 `&` 取址運算子)。回傳成功讀取的項目數。

字串處理函式

`int strlen(char *s)`：回傳字串的長度 (不含結尾的 `\0`)。

`void strcpy(char *dest, char *src)`：將來源字串複製到目的字串。

`int strcmp(char *s1, char *s2)`：比較兩個字串，回傳值為負數、零或正數，分別表示 `s1` 在字典順序中小於、等於或大於 `s2`。

`void strcat(char *dest, char *src)`：將來源字串串接至目的字串的末尾。

數學函式

`int abs(int x)`：回傳 `x` 的絕對值。

`int max(int a, int b)`：回傳 `a` 與 `b` 中的較大值。

`int min(int a, int b)`：回傳 `a` 與 `b` 中的較小值。

`int pow(int base, int exp)`：回傳 `base` 的 `exp` 次方。當 `exp` 為負數時回傳 0（整數除法的自然結果）。當 `exp` 為 0 時回傳 1。

`int sqrt(int x)`：回傳 `x` 的平方根之整數部分（即向下取整）。當 `x` 為負數時顯示執行期錯誤。

`int mod(int a, int b)`：回傳 `a` 除以 `b` 的餘數，等同於 `%` 運算子，但以函式形式提供。當 `b` 為 0 時顯示執行期錯誤。

`int rand()`：回傳一個介於 0 至 32767 之間的偽隨機整數。

`void srand(int seed)`：以指定的種子值初始化偽隨機數產生器。

記憶體與工具函式

`void memset(char *ptr, int value, int size)`：將 `ptr` 所指向的記憶體區塊之前 `size` 個位元組全部設定為 `value`。

`int sizeof_int()`：回傳 `int` 型別的位元組數（固定為 4）。

`int sizeof_char()`：回傳 `char` 型別的位元組數（固定為 1）。

`int atoi(char *s)`：將字串轉換為整數。若字串不包含有效的整數表示，回傳 0。

`void itoa(int value, char *str)`：將整數轉換為十進位字串表示，儲存至 `str` 所指向的字元陣列。

`void exit(int code)`：立即終止程式執行，回傳指定的結束碼。

2.8 註解

支援 C 風格的區塊註解 `/* ... */` 以及 C++ 風格的單行註解 `// ...`。區塊註解不支援巢狀。

2.9 前處理器

僅需支援 `#define` 指令的簡單常數定義形式，例如：

```
#define MAX_SIZE 100
#define PI_APPROX 3
```

不需支援帶參數的巨集定義（如 `#define MAX(a,b) ((a)>(b)?(a):(b))`），亦不需支援 `#include`、`#ifdef`、`#ifndef`、`#endif` 等條件編譯指令。

三、解譯器環境互動指令

解譯器啟動後應顯示歡迎訊息與版本資訊，隨後進入互動模式，以提示符 `sc>` 等候使用者輸入。使用者的輸入分為兩類：若輸入以保留指令關鍵字起始（見下表），解譯器執行對應的環境操作；否則將輸入視為 Small-C 程式碼，進行解析與執行。

所有環境指令均不區分大小寫（case-insensitive）。以下列出解譯器必須支援的環境指令及其功能說明：

3.1 程式管理指令

LOAD <filename>：從指定檔案載入 Small-C 原始碼至程式緩衝區。載入成功後顯示讀取的行數。若檔案不存在或無法開啟，應顯示適當的錯誤訊息。載入動作會覆蓋目前程式緩衝區的內容，執行前應提示使用者確認是否放棄未儲存的修改。

SAVE <filename>：將目前程式緩衝區的內容儲存至指定檔案。儲存成功後顯示寫入的行數。若檔案無法建立或寫入，應顯示適當的錯誤訊息。

LIST：列出目前程式緩衝區的完整內容，每一行前方標示行號。若緩衝區為空，應顯示提示訊息。

LIST <n>：列出程式緩衝區中第 n 行的內容。

LIST <n1>-<n2>：列出程式緩衝區中第 $n1$ 行至第 $n2$ 行的內容。

EDIT <n>：將第 n 行的內容顯示給使用者，並允許使用者輸入新內容來取代該行。若使用者直接按下 Enter 而不輸入任何內容，則保留原行不變。

DELETE <n>：刪除程式緩衝區中的第 n 行，其後各行的行號自動遞減。

DELETE <n1>-<n2>：刪除程式緩衝區中第 $n1$ 行至第 $n2$ 行的內容。

INSERT <n>：在第 n 行之前進入插入模式，使用者可逐行輸入新的程式碼，直到輸入單獨一行 `.`（句點）為止，結束插入模式。

APPEND：在程式緩衝區的末尾進入插入模式，使用者可逐行輸入新的程式碼，直到輸入單獨一行 `.`（句點）為止。

NEW：清除程式緩衝區的所有內容，同時重置所有全域變數、函式定義與執行期狀態，回到初始狀態。若目前緩衝區有未儲存的修改，應提示使用者確認。

3.2 執行與除錯指令

RUN：執行目前程式緩衝區中的程式。解譯器依序進行詞法分析、語法分析與語意檢查，若無錯誤則從 `main()` 函式開始執行。若程式中未定義 `main()` 函式，應顯示錯誤訊息。執行結束後顯示程式的回傳值（若 `main` 回傳型別為 `int`）。

RUN 指令在執行前應自動清除前一次執行所殘留的全域變數值與動態狀態，確保每次執行都從乾淨的初始狀態開始。

CHECK：對目前程式緩衝區中的程式碼進行語法與語意檢查，但不實際執行。將所有偵測到的錯誤與警告依行號順序列出。若無錯誤，顯示「No errors found」。

TRACE ON：開啟執行追蹤模式。啟用後，**RUN** 指令在執行過程中將於每個語句執行前顯示該語句所在的行號與語句內容，格式為 `[line n] <statement>`。此功能有助於學生除錯。

TRACE OFF：關閉執行追蹤模式。

VARS：顯示目前所有全域變數的名稱、型別與當前值。若為指標，同時顯示其所指向的位址值。若為陣列，顯示陣列長度及前十個元素的值（若元素超過十個，以 `...` 表示省略）。

FUNCS：列出目前已定義的所有函式名稱、回傳型別、參數列表及其起始行號。內建函式以 `[built-in]` 標示。

3.3 系統指令

HELP：顯示所有可用環境指令的摘要說明。

HELP <command>：顯示指定環境指令的詳細說明與使用範例。

ABOUT：顯示解譯器的名稱、版本號、作者資訊與修課學期。

CLEAR：清除終端機畫面（等同於執行系統的清屏操作）。

QUIT 或 **EXIT**：結束解譯器。若目前程式緩衝區有未儲存的修改，應提示使用者確認是否放棄修改後離開。

四、執行範例

以下執行範例的編排順序，刻意模擬一位學生以 Bottom-Up 方法逐步開發與測試解譯器功能的過程。Bottom-Up 方法的核心精神是：先確認最基礎的元件能正確運作，再逐層向上組合為更複雜的功能。學生在開發解譯器時，建議依循相同的順序——先實作並驗證表達式求值，再處理變數與宣告，接著是控制結構，最後才是函式與完整程式的執行。

範例 1：基本算術表達式（驗證表達式求值引擎）

此範例驗證解譯器是否能正確解析並求值整數算術表達式，包括運算子優先順序與括號的處理。這是整個解譯器最底層的功能，必須率先確認正確性。

```
=====
Small-C Interactive Interpreter v1.0
System Software Final Project, Spring 2026
=====
Type 'HELP' for a list of commands.

sc> printf("%d\n", 2 + 3 * 4);
14
sc> printf("%d\n", (2 + 3) * 4);
20
sc> printf("%d\n", 100 / 7);
14
sc> printf("%d\n", 100 % 7);
2
sc> printf("%d\n", -5 + 3);
-2
sc> printf("%d\n", 2 + 3 * 4 - 8 / 2);
10
sc> printf("%d\n", {(2 + 3) * {4 - 1}} / 5);
3
```

範例 2：關係與邏輯運算（驗證布林求值與短路邏輯）

此範例在確認算術運算正確後，向上驗證關係運算子與邏輯運算子的求值結果，這些運算子是控制結構得以運作的前提。

```
sc> printf("%d\n", 5 > 3);
1
sc> printf("%d\n", 5 < 3);
0
sc> printf("%d\n", 5 == 5);
1
sc> printf("%d\n", 5 != 5);
0
sc> printf("%d\n", 5 >= 5 && 3 < 4);
1
sc> printf("%d\n", 5 > 10 || 3 < 4);
1
sc> printf("%d\n", !(5 > 3));
0
```

範例 3：位元運算（驗證位元層級操作）

此範例驗證位元運算子的正確性，這些運算在系統程式設計中經常使用。

```
sc> printf("%d\n", 0xFF & 0x0F);
15
sc> printf("%d\n", 0xA0 | 0x05);
165
sc> printf("%d\n", 0xFF ^ 0x0F);
240
sc> printf("%d\n", ~0);
-1
sc> printf("%d\n", 1 << 8);
256
sc> printf("%d\n", 256 >> 4);
16
sc> printf("0x%x\n", (0xAB & 0xF0) | 0x0C);
0xac
```

範例 4：變數宣告與指定（驗證符號表與記憶體管理）

確認表達式求值正確後，接下來驗證變數的宣告、初始化、指定與讀取。這一層測試的是符號表的建構與查詢機制。

```
sc> int x = 10;
sc> int y = 20;
sc> int z;
sc> z = x + y;
sc> printf("x=%d, y=%d, z=%d\n", x, y, z);
x=10, y=20, z=30
sc> x += 5;
sc> y -= 3;
sc> printf("x=%d, y=%d\n", x, y);
x=15, y=17
sc> char ch = 'A';
sc> printf("ch=%c (ASCII=%d)\n", ch, ch);
ch=A (ASCII=65)
sc> VARS
int x = 15
int y = 17
int z = 30
char ch = 65 ('A')
```

範例 5：數學內建函式（驗證內建函式呼叫機制）

此範例驗證內建數學函式的呼叫與回傳值機制，同時測試函式呼叫作為表達式的一部分是否能正確運作。

```

sc> printf("abs(-42) = %d\n", abs(-42));
abs(-42) = 42
sc> printf("max(10, 25) = %d\n", max(10, 25));
max(10, 25) = 25
sc> printf("min(10, 25) = %d\n", min(10, 25));
min(10, 25) = 10
sc> printf("pow(2, 10) = %d\n", pow(2, 10));
pow(2, 10) = 1024
sc> printf("sqrt(144) = %d\n", sqrt(144));
sqrt(144) = 12
sc> printf("sqrt(150) = %d\n", sqrt(150));
sqrt(150) = 12
sc> int a = max(abs(-7), min(3, 5));
sc> printf("a = %d\n", a);
a = 7
sc> printf("pow(2,0)=%d, pow(2,1)=%d, pow(2,-1)=%d\n", pow(2,0), pow(2,1), pow(2,-1));
pow(2,0)=1, pow(2,1)=2, pow(2,-1)=0

```

範例 6：if/else 控制結構（驗證條件分支）

在變數與表達式都能正確運作後，開始測試最基本的控制結構。

```

sc> int score = 85;
sc> if (score >= 90) {
>   printf("Grade: A\n");
> } else if (score >= 80) {
>   printf("Grade: B\n");
> } else if (score >= 70) {
>   printf("Grade: C\n");
> } else {
>   printf("Grade: F\n");
> }
Grade: B
sc> int n = 17;
sc> if (n % 2 == 0) {
>   printf("%d is even\n", n);
> } else {
>   printf("%d is odd\n", n);
> }
17 is odd

```

範例 7：while 與 for 迴圈（驗證迭代結構）

此範例驗證迴圈的正確執行，包括迴圈計數器的更新與終止條件的判定。

```

sc> int i = 1;
sc> int sum = 0;
sc> while (i <= 100) {

```

```

>     sum += i;
>     i = i + 1;
> }
sc> printf("1+2+...+100 = %d\n", sum);
1+2+...+100 = 5050
sc> NEW
All cleared.
sc> int i;
sc> for (i = 1; i <= 9; i = i + 1) {
>     printf("%d * %d = %d\n", i, i, i * i);
> }
1 * 1 = 1
2 * 2 = 4
3 * 3 = 9
4 * 4 = 16
5 * 5 = 25
6 * 6 = 36
7 * 7 = 49
8 * 8 = 64
9 * 9 = 81

```

範例 8：break 與 continue（驗證迴圈控制語句）

```

sc> NEW
All cleared.
sc> int i;
sc> for (i = 1; i <= 20; i = i + 1) {
>     if (i % 3 == 0)
>         continue;
>     if (i > 15)
>         break;
>     printf("%d ", i);
> }
1 2 4 5 7 8 10 11 13 14
sc> printf("\n");

```

範例 9：陣列操作（驗證陣列配置與索引存取）

```

sc> NEW
All cleared.
sc> int arr[10];
sc> int i;
sc> for (i = 0; i < 10; i = i + 1) {
>     arr[i] = (i + 1) * 10;
> }
sc> for (i = 0; i < 10; i = i + 1) {
>     printf("arr[%d] = %d\n", i, arr[i]);
> }

```

```

arr[0] = 10
arr[1] = 20
arr[2] = 30
arr[3] = 40
arr[4] = 50
arr[5] = 60
arr[6] = 70
arr[7] = 80
arr[8] = 90
arr[9] = 100
sc> VARS
    int arr[10] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100}
    int i = 10

```

範例 10：字串操作與字串函式（驗證 char 陣列與內建字串函式）

```

sc> NEW
All cleared.
sc> char name[40];
sc> strcpy(name, "Hello");
sc> printf("name = \"%s\", length = %d\n", name, strlen(name));
name = "Hello", length = 5
sc> strcat(name, " World");
sc> printf("name = \"%s\", length = %d\n", name, strlen(name));
name = "Hello World", length = 11
sc> printf("strcmp result: %d\n", strcmp("abc", "abd"));
strcmp result: -1
sc> printf("atoi(\"12345\") = %d\n", atoi("12345"));
atoi("12345") = 12345

```

範例 11：指標基本操作（驗證取址與取值運算）

```

sc> NEW
All cleared.
sc> int x = 42;
sc> int *ptr;
sc> ptr = &x;
sc> printf("x = %d\n", x);
x = 42
sc> printf("*ptr = %d\n", *ptr);
*ptr = 42
sc> *ptr = 99;
sc> printf("x = %d\n", x);
x = 99
sc> printf("ptr points to address %d\n", ptr);
ptr points to address 1000

```

範例 12：使用程式緩衝區開發函式（展示 Bottom-Up 的完整程式開發流程）

此範例展示學生如何在互動環境中，利用程式管理指令逐步撰寫、測試與修正一個完整的程式。

```
sc> NEW
All cleared.
sc> APPEND
1> /* Bubble Sort Demo */
2> void swap(int *a, int *b) {
3>     int temp;
4>     temp = *a;
5>     *a = *b;
6>     *b = temp;
7> }
8>
9> void bubble_sort(int *arr, int n) {
10>     int i;
11>     int j;
12>     for (i = 0; i < n - 1; i = i + 1) {
13>         for (j = 0; j < n - 1 - i; j = j + 1) {
14>             if (arr[j] > arr[j + 1]) {
15>                 swap(&arr[j], &arr[j + 1]);
16>             }
17>         }
18>     }
19> }
20>
21> void print_array(int *arr, int n) {
22>     int i;
23>     for (i = 0; i < n; i = i + 1) {
24>         printf("%d ", arr[i]);
25>     }
26>     printf("\n");
27> }
28>
29> int main() {
30>     int data[8];
31>     data[0] = 64; data[1] = 25; data[2] = 12; data[3] = 22;
32>     data[4] = 11; data[5] = 90; data[6] = 45; data[7] = 31;
33>
34>     printf("Before sorting: ");
35>     print_array(data, 8);
36>
37>     bubble_sort(data, 8);
38>
39>     printf("After sorting: ");
40>     print_array(data, 8);
41>
42>     return 0;
43> }
44> .
sc> LIST
```

```

1: /* Bubble Sort Demo */
2: void swap(int *a, int *b) {
3:     int temp;
4:     temp = *a;
5:     *a = *b;
6:     *b = temp;
7: }
8:
9: void bubble_sort(int *arr, int n) {
10:     int i;
11:     int j;
12:     for (i = 0; i < n - 1; i = i + 1) {
13:         for (j = 0; j < n - 1 - i; j = j + 1) {
14:             if (arr[j] > arr[j + 1]) {
15:                 swap(&arr[j], &arr[j + 1]);
16:             }
17:         }
18:     }
19: }
20:
21: void print_array(int *arr, int n) {
22:     int i;
23:     for (i = 0; i < n; i = i + 1) {
24:         printf("%d ", arr[i]);
25:     }
26:     printf("\n");
27: }
28:
29: int main() {
30:     int data[8];
31:     data[0] = 64; data[1] = 25; data[2] = 12; data[3] = 22;
32:     data[4] = 11; data[5] = 90; data[6] = 45; data[7] = 31;
33:
34:     printf("Before sorting: ");
35:     print_array(data, 8);
36:
37:     bubble_sort(data, 8);
38:
39:     printf("After sorting: ");
40:     print_array(data, 8);
41:
42:     return 0;
43: }

```

sc> CHECK

No errors found.

sc> RUN

Before sorting: 64 25 12 22 11 90 45 31

After sorting: 11 12 22 25 31 45 64 90

Program exited with return value 0.

sc> SAVE bubble_sort.sc

Saved 43 lines to 'bubble_sort.sc'.

範例 13：遞迴函式（驗證呼叫堆疊與遞迴機制）

```
sc> NEW
All cleared.
sc> APPEND
1> int fibonacci(int n) {
2>     if (n <= 0) return 0;
3>     if (n == 1) return 1;
4>     return fibonacci(n - 1) + fibonacci(n - 2);
5> }
6>
7> int main() {
8>     int i;
9>     printf("Fibonacci sequence:\n");
10>    for (i = 0; i < 15; i = i + 1) {
11>        printf("F(%d) = %d\n", i, fibonacci(i));
12>    }
13>    return 0;
14> }
15> .
sc> RUN
Fibonacci sequence:
F(0) = 0
F(1) = 1
F(2) = 1
F(3) = 2
F(4) = 3
F(5) = 5
F(6) = 8
F(7) = 13
F(8) = 21
F(9) = 34
F(10) = 55
F(11) = 89
F(12) = 144
F(13) = 233
F(14) = 377
Program exited with return value 0.
```

範例 14：使用 TRACE 除錯（展示追蹤模式的實際用途）

```
sc> NEW
All cleared.
sc> APPEND
1> int gcd(int a, int b) {
2>     while (b != 0) {
3>         int temp;
4>         temp = b;
5>         b = a % b;
```

```

6>     a = temp;
7> }
8> return a;
9> }
10>
11> int main() {
12>     int result;
13>     result = gcd(48, 18);
14>     printf("GCD(48, 18) = %d\n", result);
15>     return 0;
16> }
17> .
sc> TRACE ON
Trace mode enabled.
sc> RUN
[line 12] int result;
[line 13] result = gcd(48, 18);
    [line 2] while (b != 0) {
        [line 4] temp = b;
        [line 5] b = a % b;
        [line 6] a = temp;
    [line 2] while (b != 0) {
        [line 4] temp = b;
        [line 5] b = a % b;
        [line 6] a = temp;
    [line 2] while (b != 0) {
        [line 4] temp = b;
        [line 5] b = a % b;
        [line 6] a = temp;
    [line 2] while (b != 0) {
        [line 8] return a;
[line 14] printf("GCD(48, 18) = %d\n", result);
GCD(48, 18) = 6
[line 15] return 0;
Program exited with return value 0.
sc> TRACE OFF
Trace mode disabled.

```

範例 15：使用 EDIT 修正程式錯誤（展示互動式開發的除錯流程）

```

sc> NEW
All cleared.
sc> APPEND
1> int main() {
2>     int i;
3>     int sum = 0;
4>     for (i = 1; i <= 10; i = i + 1) {
5>         sum += i * i;
6>     }
7>     printf("Sum of squares: %d\n", sum);

```

```

8>     return 0;
9> }
10> .
sc> RUN
Sum of squares: 385
Program exited with return value 0.
sc> LIST 5
5:         sum += i * i;
sc> EDIT 5
5:         sum += i * i * i;
sc> LIST 4-6
4:     for (i = 1; i <= 10; i = i + 1) {
5:         sum += i * i * i;
6:     }
sc> EDIT 7
7:     printf("Sum of cubes: %d\n", sum);
sc> RUN
Sum of cubes: 3025
Program exited with return value 0.

```

範例 16：錯誤偵測與回報（驗證錯誤處理機制）

```

sc> int x = 10 / 0;
Runtime error: division by zero.
sc> int arr[5];
sc> arr[10] = 99;
Runtime error: array index out of bounds (index 10, size 5).
sc> printf("%d\n", sqrt(-1));
Runtime error: sqrt() argument must be non-negative.
sc> int y = ;
Syntax error: unexpected token ';', expected expression.
sc> if (x > ) { }
Syntax error: unexpected token ')', expected expression.
sc> APPEND
1> int main() {
2>     int x = 10;
3>     printf("%d\n", x)
4>     return 0;
5> }
6> .
sc> CHECK
Error at line 3: expected ';' after expression statement.
1 error(s) found.

```

範例 17：FUNCS 指令展示（驗證函式資訊查詢）

```

sc> LOAD bubble_sort.sc
Loaded 43 lines from 'bubble_sort.sc'.

```

```

sc> CHECK
No errors found.
sc> FUNCS
void swap(int *a, int *b)           line 2
void bubble_sort(int *arr, int n)   line 9
void print_array(int *arr, int n)   line 21
int main()                          line 29
--- built-in functions ---
int putchar(int ch)                 [built-in]
int getchar()                       [built-in]
void printf(char *fmt, ...)         [built-in]
void puts(char *s)                  [built-in]
int scanf(char *fmt, ...)           [built-in]
int strlen(char *s)                 [built-in]
void strcpy(char *dest, char *src)  [built-in]
int strcmp(char *s1, char *s2)      [built-in]
void strcat(char *dest, char *src)  [built-in]
int abs(int x)                      [built-in]
int max(int a, int b)               [built-in]
int min(int a, int b)               [built-in]
int pow(int base, int exp)          [built-in]
int sqrt(int x)                     [built-in]
int mod(int a, int b)               [built-in]
int rand()                          [built-in]
void srand(int seed)                [built-in]
void memset(char *ptr, int val, int n) [built-in]
int sizeof_int()                    [built-in]
int sizeof_char()                   [built-in]
int atoi(char *s)                   [built-in]
void itoa(int val, char *str)       [built-in]
void exit(int code)                 [built-in]

```

範例 18：完整的互動式程式開發過程（綜合展示）

此範例展示從空白狀態開始，使用 Bottom-Up 方法逐步發展一個質數篩選程式的完整過程。

```

sc> NEW
All cleared.

sc> /* Step 1: 先在互動模式下測試核心邏輯 */
sc> int n = 29;
sc> int i;
sc> int is_prime = 1;
sc> for (i = 2; i * i <= n; i = i + 1) {
>     if (n % i == 0) {
>         is_prime = 0;
>         break;
>     }
> }
sc> printf("%d is prime: %d\n", n, is_prime);
29 is prime: 1

```

```

sc> /* Step 2: 確認邏輯正確，現在包裝成函式寫入緩衝區 */
sc> NEW
All cleared.
sc> APPEND
1> int is_prime(int n) {
2>     int i;
3>     if (n < 2) return 0;
4>     for (i = 2; i * i <= n; i = i + 1) {
5>         if (n % i == 0) return 0;
6>     }
7>     return 1;
8> }
9>
10> int main() {
11>     int i;
12>     int count = 0;
13>     printf("Prime numbers from 2 to 100:\n");
14>     for (i = 2; i <= 100; i = i + 1) {
15>         if (is_prime(i)) {
16>             printf("%d ", i);
17>             count = count + 1;
18>         }
19>     }
20>     printf("\nTotal: %d primes\n", count);
21>     return 0;
22> }
23> .
sc> RUN
Prime numbers from 2 to 100:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
Total: 25 primes
Program exited with return value 0.
sc> SAVE primes.sc
Saved 22 lines to 'primes.sc'.
sc> QUIT
Goodbye.

```

五、繳交項目

學生於第十六週上課日前繳交以下項目，所有檔案以壓縮檔（.zip）格式上傳至課程教學平台：

5.1 **原始碼**：解譯器的完整 Python 3 原始碼，應包含適當的檔案分割與模組化設計（建議至少分為 `main.py`、`lexer.py`、`parser.py`、`interpreter.py`、`symtable.py`、`memory.py`、`builtins.py`、`repl.py`），並附上 `requirements.txt`（若有使用第三方套件）以及 `README.md`，說明執行方式與相依套件。解譯器應能以 `python3 main.py` 指令直接啟動。

5.2 **測試程式集**：至少十個 Small-C 測試程式（.sc 檔案），涵蓋以下測試類別：基本算術與變數操作（至少 2 個）、控制結構（至少 2 個）、函式定義與遞迴呼叫（至少 2 個）、陣列與指標操作（至少 2 個）、錯誤處理驗證（至少 2 個，用以展示解譯器對語法錯誤與執行期錯誤的偵測與回報能力）。每個測試程式應附上對應的預期輸出檔案（.expected）。

5.3 **專題報告**：一份 PDF 格式的技術報告，篇幅不少於十頁（不含附錄），內容須包含以下章節：系統架構設計說明（含模組劃分圖與資料流程圖）、各模組的設計決策與實作細節、符號表的資料結構設計、記憶體模型的模擬方式、測試策略與測試結果分析、開發過程中遭遇的主要困難及其解決方案、以及各組員的工作分配與貢獻說明（若為團隊作業）。

5.4 **展示影片**：一段五至十分鐘的螢幕錄製影片（.mp4 格式），展示解譯器的啟動畫面、環境指令操作、以及至少三個測試程式的載入與執行過程。

六、評分準則

本專題總分 100 分，各評分項目的配分與評量標準如下：

6.1 **詞法分析與語法分析**（25 分）：正確辨識所有 Small-C 的 token 類型（10 分）。正確解析變數宣告、表達式、控制結構與函式定義（10 分）。語法錯誤的偵測與有意義的錯誤訊息回報（5 分）。

6.2 **語意分析與執行引擎**（30 分）：全域與區域變數的正確管理，包括作用域規則（6 分）。算術、關係、邏輯、位元運算的正確求值及運算子優先順序（6 分）。控制結構（if/else、while、for、do/while、break、continue）的正確執行（6 分）。函式呼叫、參數傳遞、回傳值與遞迴的正確處理（6 分）。陣列與指標操作的正確執行，包括指標算術（6 分）。

6.3 **互動環境**（20 分）：REPL 的基本運作，包括多行輸入偵測與持續性狀態管理（5 分）。程式管理指令（LOAD、SAVE、LIST、EDIT、DELETE、INSERT、APPEND、NEW）的完整實作（8 分）。執行與除錯指令（RUN、CHECK、TRACE、VARS、FUNCS）的完整實作（5 分）。系統指令（HELP、ABOUT、CLEAR、QUIT/EXIT）的完整實作（2 分）。

6.4 **程式品質與文件**（15 分）：原始碼的模組化程度、命名慣例、縮排風格與註解品質（5 分）。測試程式集的涵蓋範圍與測試設計品質（5 分）。專題報告的完整性、清晰度與技術深度（5 分）。

6.5 **加分項目**（最高額外 15 分）：switch/case 語句的支援（5 分）。執行期錯誤的完善處理，包括除以零偵測、陣列越界檢查、空指標取值偵測（5 分）。#define 常數替換的完整實作（5 分）。

七、關於使用生成式人工智慧工具

本專題允許並鼓勵學生使用生成式人工智慧工具（如 ChatGPT、Claude、GitHub Copilot 等）輔助開發。人工智慧工具在現代軟體工程中已是不可或缺的生產力工具，學習如何有效地運用這些工具本身即為重要的專業能力。

然而，使用人工智慧工具並不免除學生理解自己所繳交程式碼的責任。評分者將以教師另行提供的大量測試程式與互動操作腳本，對每一組繳交的解譯器進行自動化與手動的綜合測試。這些測試案例將涵蓋本作業說明中所有語言特性、環境指令與邊角情況。任何未通過測試的功能將依評分準則扣分，無論該功能是由學生手工撰寫或由人工智慧工具產生。

此外，展示時評分者將隨機提問程式碼的設計邏輯、模組間的協作方式、以及特定功能的實作細節。無法合理解釋自己所繳交程式碼的學生，將被視為未達成學習目標，評分將按比例調降。

建議學生在專題報告中如實說明人工智慧工具的使用方式與範圍，包括哪些模組借助了工具的輔助、工具產生的程式碼進行了哪些修改與驗證、以及在哪些環節遭遇了工具的局限性而需要自行設計解決方案。這些紀錄不會影響評分，反而有助於評分者理解學生的學習過程。

八、參考資料

以下為學生在開發過程中可參考的線上資源與文獻。第一類為 Small-C 語言的原始定義與歷史資料，第二類為解譯器設計的技術參考。

Small-C 語言相關

Ron Cain 於 1980 年發表的原始 Small-C 編譯器原始碼與說明文章，可於多個線上典藏站點取得全文。James Hendrix 所著的《The Small-C Handbook》對語言規範與編譯器實作均有詳盡的闡述，為本語言子集最具權威性的參考來源。此外，GitHub 上存有多個 Small-C 編譯器的現代重製版本，學生可以參考其語法定義與設計思路。

解譯器設計相關

Robert Nystrom 所著的《Crafting Interpreters》是一本可在線上免費閱讀的解譯器教科書，以 Java 與 C 語言分別實作了一個完整的程式語言解譯器，其中關於遞迴下降解析與 tree-walking interpretation 的章節與本專題高度相關。

Jack Crenshaw 所著的《Let's Build a Compiler》為一系列線上教學文章，以極度精簡的方式逐步建構一個編譯器，其中關於遞迴下降解析與表達式求值的教學方式非常直觀，適合作為入門閱讀。

Ruslan Spivak 所著的《Let's Build a Simple Interpreter》為一系列以 Python 實作的解譯器教學部落格文章，從最基本的算術表達式求值器逐步擴展至支援變數、條件與函式的完整解譯器，其 Python 實作風格與本專題的技術棧最為接近，強烈建議參閱。

References

Beck, L. L. (1996). *System software: An introduction to systems programming* (3rd ed.). Addison-Wesley.

Cain, R. (1980). A small C compiler. *Dr. Dobb's Journal of Computer Calisthenics & Orthodontia*, 5(45), 5–19.

Crenshaw, J. W. (1988–1995). *Let's build a compiler*. <https://compilers.iecc.com/crenshaw/>

Hendrix, J. E. (1984). *The Small-C handbook*. Reston Publishing.

Nystrom, R. (2021). *Crafting interpreters*. Genever Benning. <https://craftinginterpreters.com/>

Spivak, R. (2015–2017). *Let's build a simple interpreter*. <https://ruslanspivak.com/lbasi-part1/>